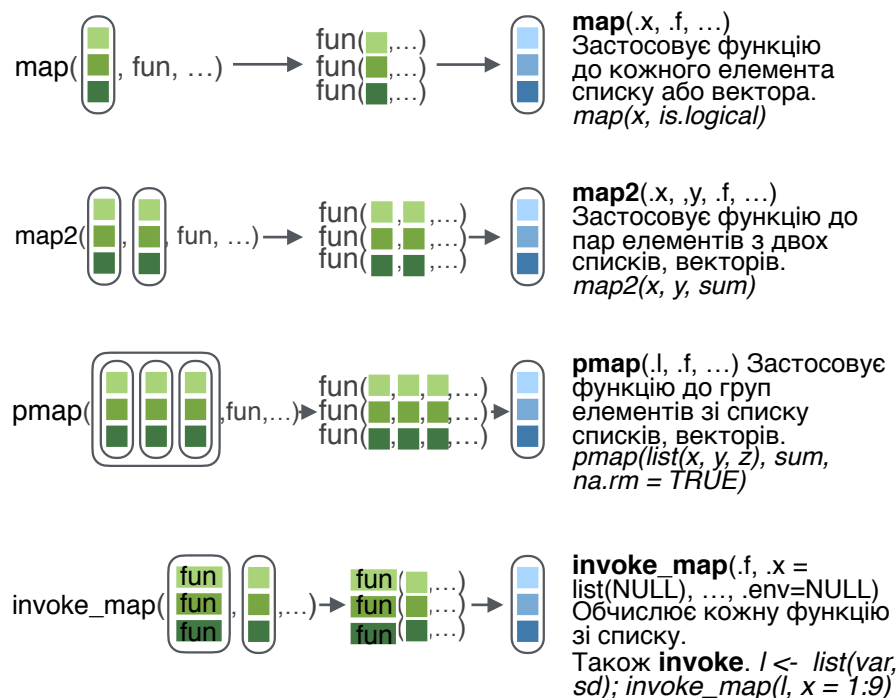


Застосування функцій з purrr : : ШПАРГАЛКА



Застосування функцій

Мар-функції ітеративно застосовують функції до кожного елементу списку або вектора.



lmap(x, f, ...) Застосовує f до спискових елементів списку, вектора.
imap(x, f, ...) Застосовує f до елементів списку, вектора та його індексу.

РЕЗУЛЬТАТ

map(), map2(), pmap(), lmap та **invoke_map** повертають список. Використовуйте версії з суфіксом для отримання результату певного типу, наприклад **map2_chr, pmap_lgl, i** т.д.

Використовуйте **walk, walk2** та **pwalk** для здійснення побічних дій. Усі невидимо повертають вхідні дані.

функція	результат
map	список
map_chr	символьний вектор
map_dbl	числовий вектор
map_dfc	data frame (з'єд. по стовпц.)
map_dfr	data frame (з'єд. по рядкам)
map_int	цілочисельний вектор
map_lgl	логічний вектор
walk	здійснює побічні дії, невидимо повертає вхідні дані

СКОРОЧЕННЯ - всередині функцій purrr:

"name" стає **function(x) x[["name"]]**, напр. *map(l, "a")* витягує a з кожного елемента l

~ .x .y стає **function(x, y) .x .y**, напр. *map2(l, p, ~ .x + .y)* стає *map2(l, p, function(l, p) l + p)*

~ . стає **function(x) x**, напр. *map(l, ~ 2 + .)* стає *map(l, function(x) 2 + x)*

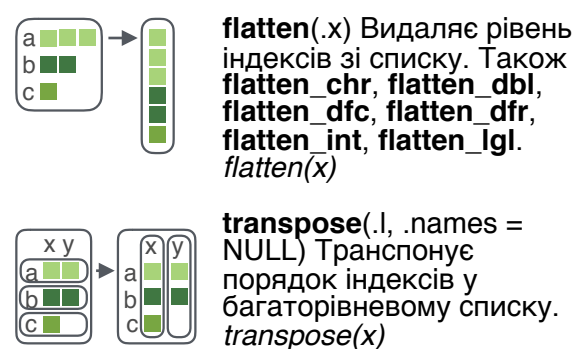
~ ..1 ..2 etc стає **function(..1, ..2, etc) ..1 ..2 etc** напр. *pmap(list(a, b, c), ~ ..3 + ..1 - ..2)* стає *pmap(list(a, b, c), function(a, b, c) c + a - b)*

Робота зі списками

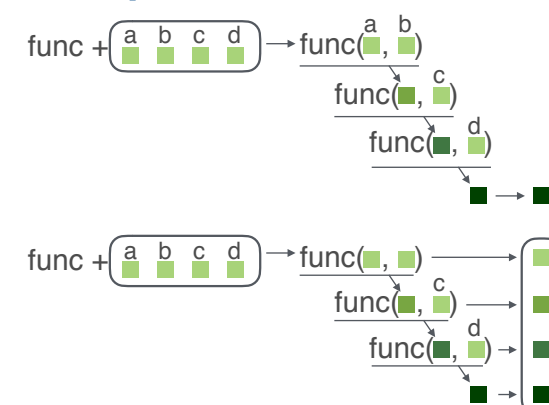
ФІЛЬТРУВАННЯ СПИСКІВ



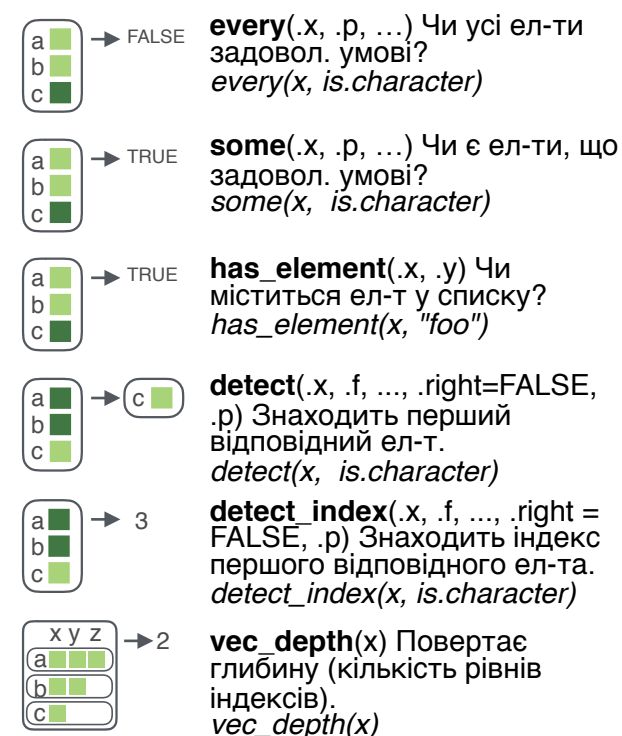
ФОРМАТУВАННЯ СПИСКІВ



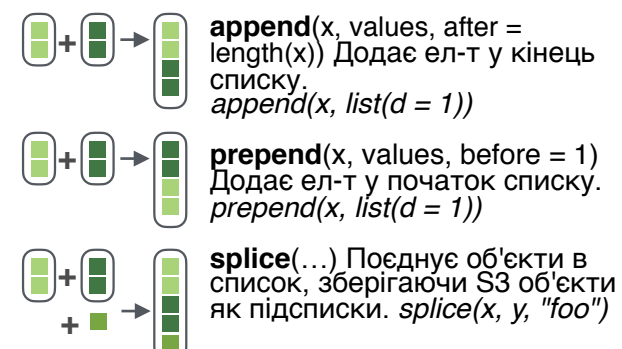
Згортка списків



ПІДСУМОВУВАННЯ СПИСКІВ



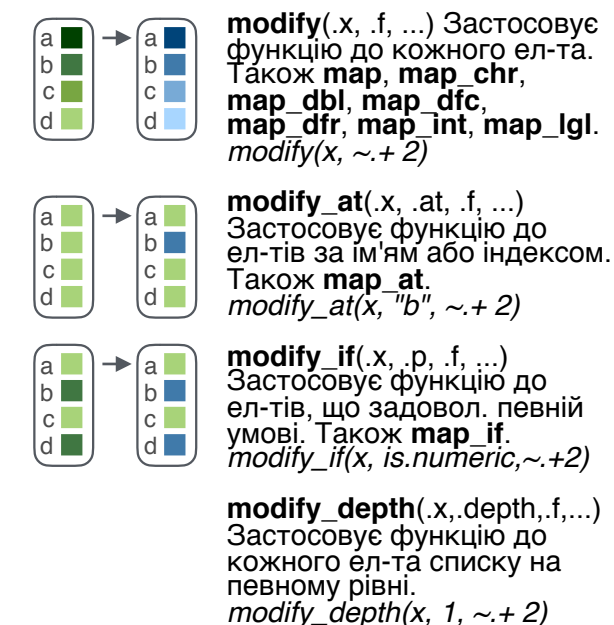
З'ЄДНАННЯ СПИСКІВ



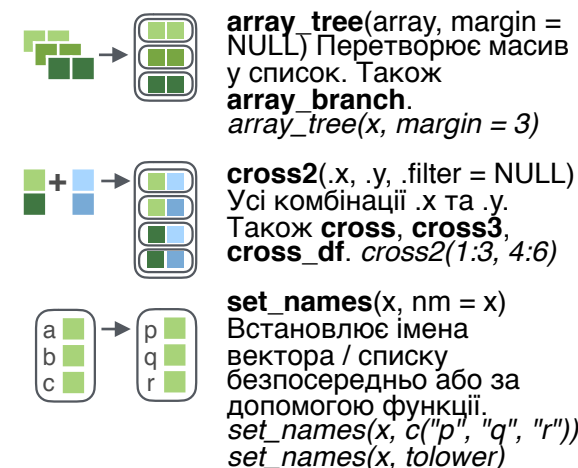
reduce(x, .f, ..., .init) Застосовує функцію рекурсивно до кожного ел-та списку, вектора. Також **reduce_right, reduce2, reduce2_right, reduce(x, sum)**

accumulate(x, .f, ..., .init) Як **reduce()**, але повертає проміжні результати. Також **accumulate_right, accumulate(x, sum)**

ПЕРЕТВОРЕННЯ СПИСКІВ



РОБОТА ЗІ СПИСКАМИ



Модифікація функцій

compose() Створює композицію функцій.

lift() Змінює тип вхідних даних функції. Також **lift_d, lift_dv, lift_ld, lift_lv, lift_vd, lift_vl**

rerun() Незалежно виконує вираз n разів.

negate() Створює заперечення предикатної функції (можна використовувати в конвеєрі!)

partial() Змінює функцію, задаючи певні параметри.

safely() Змінює функцію, щоб повертала список рез-тів та помилок.

quietly() Змінює функцію, щоб повертала список рез-тів, виведення, повідомлень і попереджень.

possibly() Змінює функцію, щоб повертала значення за замовчуванням при появі помилки (замість помилки).

Вкладені дані

Вкладений data frame зберігає окремі таблиці всередині клітинок більшої упорядковуючої таблиці

вкладений data frame

Species	data
setosa	<tibble [50 x 4]>
versicolor	<tibble [50 x 4]>
virginica	<tibble [50 x 4]>

n_iris

Використовуйте вкладений data frame для:

- збереження відносин між спостереженнями та підмножинами даних

- маніпулювання відразу декількома підтаблицями з **purrr** функціями **map()**, **map2()** або **pmap()**.

Використовуйте двоетапний процес для створення вкладеного data frame:

- Згрупуйте data frame у групи за допомогою **dplyr::group_by()**
- Використовуйте **nest()** для створення вкладеного data frame з одним рядком на групу.

Species	S.L	S.W	P.L	P.W
setosa	5.1	3.5	1.4	0.2
setosa	4.9	3.0	1.4	0.2
setosa	4.7	3.2	1.3	0.2
setosa	4.6	3.1	1.5	0.2
setosa	5.0	3.6	1.4	0.2
versi	7.0	3.2	4.7	1.4
versi	6.4	3.2	4.5	1.5
versi	6.9	3.1	4.9	1.5
versi	5.5	2.3	4.0	1.3
versi	6.5	2.8	4.6	1.5
virgini	6.3	3.3	6.0	2.5
virgini	5.8	2.7	5.1	1.9
virgini	7.1	3.0	5.9	2.1
virgini	6.3	2.9	5.6	1.8
virgini	6.5	3.0	5.8	2.2

Species	S.L	S.W	P.L	P.W
setosa	5.1	3.5	1.4	0.2
setosa	4.9	3.0	1.4	0.2
setosa	4.7	3.2	1.3	0.2
setosa	4.6	3.1	1.5	0.2
setosa	5.0	3.6	1.4	0.2
versi	7.0	3.2	4.7	1.4
versi	6.4	3.2	4.5	1.5
versi	6.9	3.1	4.9	1.5
versi	5.5	2.3	4.0	1.3
versi	6.5	2.8	4.6	1.5
virgini	6.3	3.3	6.0	2.5
virgini	5.8	2.7	5.1	1.9
virgini	7.1	3.0	5.9	2.1
virgini	6.3	2.9	5.6	1.8
virgini	6.5	3.0	5.8	2.2

```
n_iris <- iris %>% group_by(Species) %>% nest()
```

```
tidyr::nest(data, ..., .key = data)
```

Згортає групи в клітинки у вигляді data frame (для згруп. даних).

Розгортає вкладений data frame за допомогою **unnest()**:

```
n_iris %>% unnest()
```

```
tidyr::unnest(data, ..., .drop = NA, .id=NULL, .sep=NULL)
```

Розгортає вкладений data frame.

Species	data
setos	<tibble [50x4]>
versi	<tibble [50x4]>
virgini	<tibble [50x4]>

Species	S.L	S.W	P.L	P.W
setosa	5.1	3.5	1.4	0.2
setosa	4.9	3.0	1.4	0.2
setosa	4.7	3.2	1.3	0.2
setosa	4.6	3.1	1.5	0.2
versi	7.0	3.2	4.7	1.4
versi	6.4	3.2	4.5	1.5
versi	6.9	3.1	4.9	1.5
versi	5.5	2.3	4.0	1.3
virgini	6.3	3.3	6.0	2.5
virgini	5.8	2.7	5.1	1.9
virgini	7.1	3.0	5.9	2.1
virgini	6.3	2.9	5.6	1.8



Процес роботи зі СПИСКОВИМИ СТОВПЧИКАМИ

Вкладений data frame використовує **СПИСКОВИЙ СТОВПЧИК**, тобто список, який зберігається як стовпчик в data frame. Типовий **робочий процес** зі списковими стовпчиками:



1 Створення СПИСКОВОГО СТОВПЧИКА

Species	S.L	S.W	P.L	P.W
setosa	5.1	3.5	1.4	0.2
setosa	4.9	3.0	1.4	0.2
setosa	4.7	3.2	1.3	0.2
setosa	4.6	3.1	1.5	0.2
versi	7.0	3.2	4.7	1.4
versi	6.4	3.2	4.5	1.5
versi	6.9	3.1	4.9	1.5
versi	5.5	2.3	4.0	1.3
virgini	6.3	3.3	6.0	2.5
virgini	5.8	2.7	5.1	1.9
virgini	7.1	3.0	5.9	2.1
virgini	6.3	2.9	5.6	1.8

```
n_iris <- iris %>%  
group_by(Species) %>%  
nest()
```

2 Робота зі СПИСКОВИМИ СТОВПЧИКАМИ

Species	data	model
setosa	<tibble [50x4]>	<S3: lm>
versi	<tibble [50x4]>	<S3: lm>
virgini	<tibble [50x4]>	<S3: lm>

```
mod_fun <- function(df)  
lm(Sepal.Length ~ ., data = df)  
  
m_iris <- n_iris %>%  
mutate(model = map(data, mod_fun))
```

3 Спрощення СПИСКОВОГО СТОВПЧИКА

Species	beta
setos	2.35
versi	1.89
virgini	0.69

```
b_fun <- function(mod)  
coefficients(mod)[1]  
  
m_iris %>% transmute(Species,  
beta = map_dbl(model,  
b_fun))
```

1. СТВОРЕННЯ СПИСКОВОГО СТОВПЧИКА - створюйте за допом. функцій з пакетів **tibble** та **dplyr**, а також **tidyr::nest()**

tibble::tribble(...)

Створює списковий стовпчик за необхідністю

```
tribble( ~max, ~seq,  
3, 1:3,  
4, 1:4,  
5, 1:5)
```

max	seq
3	<int [3]>
4	<int [4]>
5	<int [5]>

tibble::tibble(...)

Зберігає список як списковий стовпчик.

```
tibble(max = c(3, 4, 5), seq = list(1:3, 1:4, 1:5))
```

tibble::enframe(x, name="name", value="value")

Конвертує багаторівневий список у tibble зі списковими стовпчиками.

```
enframe(list('3'=1:3, '4'=1:4, '5'=1:5), 'max', 'seq')
```

dplyr::mutate(.data, ...) Також **transmute()**

Повертає списковий стовпчик, коли результат - список.

```
mtcars %>% mutate(seq = map(cyl, seq))
```

dplyr::summarise(.data, ...)

Повертає списковий стовпчик, коли результат створений з **list()**.

```
mtcars %>% group_by(cyl) %>%  
summarise(q = list(quantile(mpg)))
```

2. РОБОТА ЗІ СПИСКОВИМИ СТОВПЧИКАМИ - Використовуйте функції **purrr map()**, **map2()** та **pmap()** для поелементного застосування функції з поверненням результату в списковий стовпчик. **walk()**, **walk2()** та **pwalk()** працюють аналогічно, але здійснюють побічні дії.

purrr::map(x, .f, ...)

Застосовує .f поелементно до .x як .f(.x)

```
n_iris %>% mutate(n = map(data, dim))
```

purrr::map2(x, .y, .f, ...)

Застосовує .f поелементно до .x та .y як .f(.x, .y)

```
m_iris %>% mutate(n = map2(data, model, list))
```

purrr::pmap(.l, .f, ...)

Застосовує .f поелементно до векторів з .l

```
m_iris %>%  
mutate(n = pmap(list(data, model, data), list))
```

data	fun	result
<tibble [50x4]>	fun(<tibble [50x4]>, ...)	result 1
<tibble [50x4]>	fun(<tibble [50x4]>, ...)	result 2
<tibble [50x4]>	fun(<tibble [50x4]>, ...)	result 3

data	model	fun	result
<tibble [50x4]>	<S3: lm>	fun(<tibble [50x4]>, <S3: lm>, ...)	result 1
<tibble [50x4]>	<S3: lm>	fun(<tibble [50x4]>, <S3: lm>, ...)	result 2
<tibble [50x4]>	<S3: lm>	fun(<tibble [50x4]>, <S3: lm>, ...)	result 3

data	model	funcs	result
<tibble [50x4]>	<S3: lm>	coef	result 1
<tibble [50x4]>	<S3: lm>	AIC	result 2
<tibble [50x4]>	<S3: lm>	BIC	result 3

3. СПРОЩЕННЯ СПИСКОВОГО СТОВПЧИКА (до звичайного стовпчика)

Використовуйте функції **purrr map_lgl()**, **map_int()**, **map_dbl()**, **map_chr()**, а також **tidyr::unnest()** для перетворення спискового стовпчика на звичайний.

purrr::map_lgl(x, .f, ...) Застосовує .f поелементно до .x, повертає логічний вектор

```
n_iris %>% transmute(n = map_lgl(data, is.matrix))
```

purrr::map_int(x, .f, ...) Застосовує .f поелементно до .x, повертає цілочис. вектор

```
n_iris %>% transmute(n = map_int(data, nrow))
```

purrr::map_dbl(x, .f, ...) Застосовує .f поелементно до .x, повертає числовий вектор

```
n_iris %>% transmute(n = map_dbl(data, nrow))
```

purrr::map_chr(x, .f, ...) Застосовує .f поелементно до .x, повертає симв. вектор

```
n_iris %>% transmute(n = map_chr(data, nrow))
```